```
================================================================================
 Anomie's SPC700 Doc (only the instructions)
 $Revision: 1162 $
 $Date: 2009-07-19 21:25:30 -0400 (Sun, 19 Jul 2009) $
 <anomie@users.sourceforge.net>
================================================================================

OPCODES
=======

In the below,
 (N) means the byte (or word when used with YA) at N.
 [N] means the word address at N.
 d   is a direct-page address.
 a   is an absolute address.
 m.b indicates that the 16-bit operand specifies a 13-bit absolute address with
     the high 3 bits indicating which bit of the addressed byte is to be
     affected.
 d.# indicates that only bit # of the byte at direct page address d is to be
     affected.

Operands are encoded little endian, with multiple operands stored last to
first. For example, "OP A, B" is stored in memory as "OP B A". Mnemonics are
represented as "OP dest, src" where applicable. However, there are a few
exceptions:
 * BBC, BBS, CBNE, and DBNZ all store the 'r' as the second byte. For example,
   BBC $01.0, $02 would be stored as "13 01 02".

DAA and DAS depend on C and H: First, if A>0x99 or Carry/Borrow, add/sub 0x60
and set Carry/Borrow. Then if (A&0x0f)>9 or HalfCarry/HalfBorrow, add/sub 0x06
(but don't change HalfCarry/Borrow).

DIV has some interesting corner cases. ZN are set based on A. V is set if
YA/X>$FF (so the result won't fit in A). H is odd, it seems to get set based on
X&$F<=Y&$F. The result is correct as long as YA/X<0x200, otherwise Y and A are
not helpful. An algorithm:
  uint17 yva=YA
  uint17 x=X<<9
  loop 9 times {
      ROL yva
      if(yva>=x) yva=yva XOR 1
      if(yva&1) yva-=x       // remember, clip to 17 bits
  }
  yva => Y, A, and V flag as YYYYYYYY V AAAAAAAA

Execution and instructions will wrap from $ffff to $0000. Direct page accesses
will wrap within the direct page (page $00 or $01, depending on the P flag).
The stack is always in page 1, and will wrap as such always.

Most of the MOV instructions targeting memory actually include a read cycle on
the destination address in addition to the expected write cycle. For example,
"MOV $ff, #$00" will read from $ff at some point, and therefore will reset
T2OUT. OTOH, "MOV $ff, $00" won't. MOVW does a read on the low byte of the
destination only. Note that none of this applies to SET1, CLR1, OR, or any
other opcode, since those are all RMW instructions.

--------------------------------------------------------------------------------
Mnemonic            Code Bytes  Cyc  Operation                        NVPBHIZC
--------------------------------------------------------------------------------
  ADC   (X), (Y)     99    1     5   (X) = (X)+(Y)+C                  NV..H.ZC
  ADC   A, #i        88    2     2   A = A+i+C                        NV..H.ZC
  ADC   A, (X)       86    1     3   A = A+(X)+C                      NV..H.ZC
  ADC   A, [d]+Y     97    2     6   A = A+([d]+Y)+C                  NV..H.ZC
  ADC   A, [d+X]     87    2     6   A = A+([d+X])+C                  NV..H.ZC
  ADC   A, d         84    2     3   A = A+(d)+C                      NV..H.ZC
  ADC   A, d+X       94    2     4   A = A+(d+X)+C                    NV..H.ZC
  ADC   A, !a        85    3     4   A = A+(a)+C                      NV..H.ZC
  ADC   A, !a+X      95    3     5   A = A+(a+X)+C                    NV..H.ZC
  ADC   A, !a+Y      96    3     5   A = A+(a+Y)+C                    NV..H.ZC
  ADC   dd, ds       89    3     6   (dd) = (dd)+(d)+C                NV..H.ZC
```

```
ADC   d, #i        98    3     5   (d) = (d)+i+C                     NV..H.ZC

ADDW  YA, d        7A    2     5   YA  = YA + (d), H on high byte    NV..H.ZC

AND   (X), (Y)     39    1     5   (X) = (X) & (Y)                   N.....Z.
AND   A, #i        28    2     2   A = A & i                         N.....Z.
AND   A, (X)       26    1     3   A = A & (X)                       N.....Z.
AND   A, [d]+Y     37    2     6   A = A & ([d]+Y)                   N.....Z.
AND   A, [d+X]     27    2     6   A = A & ([d+X])                   N.....Z.
AND   A, d         24    2     3   A = A & (d)                       N.....Z.
AND   A, d+X       34    2     4   A = A & (d+X)                     N.....Z.
AND   A, !a        25    3     4   A = A & (a)                       N.....Z.
AND   A, !a+X      35    3     5   A = A & (a+X)                     N.....Z.
AND   A, !a+Y      36    3     5   A = A & (a+Y)                     N.....Z.
AND   dd, ds       29    3     6   (dd) = (dd) & (ds)                N.....Z.
AND   d, #i        38    3     5   (d) = (d) & i                     N.....Z.

AND1  C, /m.b      6A    3     4   C = C & ~(m.b)                    .......C
AND1  C, m.b       4A    3     4   C = C & (m.b)                     .......C

ASL   A            1C    1     2   Left shift A: high->C, 0->low     N.....ZC
ASL   d            0B    2     4   Left shift (d) as above           N.....ZC
ASL   d+X          1B    2     5   Left shift (d+X) as above         N.....ZC
ASL   !a           0C    3     5   Left shift (a) as above           N.....ZC

BBC   d.0, r       13    3    5/7  PC+=r  if d.0 == 0                ........
BBC   d.1, r       33    3    5/7  PC+=r  if d.1 == 0                ........
BBC   d.2, r       53    3    5/7  PC+=r  if d.2 == 0                ........
BBC   d.3, r       73    3    5/7  PC+=r  if d.3 == 0                ........
BBC   d.4, r       93    3    5/7  PC+=r  if d.4 == 0                ........
BBC   d.5, r       B3    3    5/7  PC+=r  if d.5 == 0                ........
BBC   d.6, r       D3    3    5/7  PC+=r  if d.6 == 0                ........
BBC   d.7, r       F3    3    5/7  PC+=r  if d.7 == 0                ........

BBS   d.0, r       03    3    5/7  PC+=r  if d.0 == 1                ........
```

```
BBS   d.1, r       23    3    5/7  PC+=r  if d.1 == 1                ........
BBS   d.2, r       43    3    5/7  PC+=r  if d.2 == 1                ........
BBS   d.3, r       63    3    5/7  PC+=r  if d.3 == 1                ........
BBS   d.4, r       83    3    5/7  PC+=r  if d.4 == 1                ........
BBS   d.5, r       A3    3    5/7  PC+=r  if d.5 == 1                ........
BBS   d.6, r       C3    3    5/7  PC+=r  if d.6 == 1                ........
BBS   d.7, r       E3    3    5/7  PC+=r  if d.7 == 1                ........

BCC   r            90    2    2/4  PC+=r  if C == 0                 ........
BCS   r            B0    2    2/4  PC+=r  if C == 1                 ........
BEQ   r            F0    2    2/4  PC+=r  if Z == 1                 ........
BMI   r            30    2    2/4  PC+=r  if N == 1                 ........
BNE   r            D0    2    2/4  PC+=r  if Z == 0                 ........
BPL   r            10    2    2/4  PC+=r  if N == 0                 ........
BVC   r            50    2    2/4  PC+=r  if V == 0                 ........
BVS   r            70    2    2/4  PC+=r  if V == 1                 ........
BRA   r            2F    2     4   PC+=r                            ........

BRK                0F    1     8   Push PC and Flags, PC = [$FFDE]  ...1.0..

CALL  !a           3F    3     8   (SP--)=PCh, (SP--)=PCl, PC=a     ........

CBNE  d+X, r       DE    3    6/8  CMP A, (d+X) then BNE             ........
CBNE  d, r         2E    3    5/7  CMP A, (d) then BNE               ........

CLR1  d.0          12    2     4   d.0 = 0                          ........
CLR1  d.1          32    2     4   d.1 = 0                          ........
CLR1  d.2          52    2     4   d.2 = 0                          ........
CLR1  d.3          72    2     4   d.3 = 0                          ........
CLR1  d.4          92    2     4   d.4 = 0                          ........
CLR1  d.5          B2    2     4   d.5 = 0                          ........
CLR1  d.6          D2    2     4   d.6 = 0                          ........
CLR1  d.7          F2    2     4   d.7 = 0                          ........

CLRC               60    1     2   C = 0                            .......0
```

CLRP		20	1	2	P = 0	..0.....
CLRV		E0	1	2	V = 0, H = 0	.0..0...
CMP	(X), (Y)	79	1	5	(X) - (Y)	N.....ZC
CMP	A, #i	68	2	2	A - i	N.....ZC
CMP	A, (X)	66	1	3	A - (X)	N.....ZC
CMP	A, [d]+Y	77	2	6	A - ([d]+Y)	N.....ZC
CMP	A, [d+X]	67	2	6	A - ([d+X])	N.....ZC
CMP	A, d	64	2	3	A - (d)	N.....ZC
CMP	A, d+X	74	2	4	A - (d+X)	N.....ZC
CMP	A, !a	65	3	4	A - (a)	N.....ZC
CMP	A, !a+X	75	3	5	A - (a+X)	N.....ZC
CMP	A, !a+Y	76	3	5	A - (a+Y)	N.....ZC
CMP	X, #i	C8	2	2	X - i	N.....ZC
CMP	X, d	3E	2	3	X - (d)	N.....ZC
CMP	X, !a	1E	3	4	X - (a)	N.....ZC
CMP	Y, #i	AD	2	2	Y - i	N.....ZC
CMP	Y, d	7E	2	3	Y - (d)	N.....ZC
CMP	Y, !a	5E	3	4	Y - (a)	N.....ZC
CMP	dd, ds	69	3	6	(dd) - (ds)	N.....ZC
CMP	d, #i	78	3	5	(d) - i	N.....ZC
CMPW	YA, d	5A	2	4	YA - (d)	N.....ZC
DAA	A	DF	1	3	decimal adjust for addition	N.....ZC
DAS	A	BE	1	3	decimal adjust for subtraction	N.....ZC
DBNZ	Y, r	FE	2	4/6	Y-- then JNZ	....
DBNZ	d, r	6E	3	5/7	(d)-- then JNZ	....
DEC	A	9C	1	2	A--	N.....Z.
DEC	X	1D	1	2	X--	N.....Z.
DEC	Y	DC	1	2	Y--	N.....Z.
DEC	d	8B	2	4	(d)--	N.....Z.
DEC	d+X	9B	2	5	(d+X)--	N.....Z.
DEC	!a	8C	3	5	(a)--	N.....Z.
DECW	d	1A	2	6	Word (d)--	N.....Z.
DI		C0	1	3	I = 0	.....0..
DIV	YA, X	9E	1	12	A=YA/X, Y=mod(YA,X)	NV..H.Z.
EI		A0	1	3	I = 1	.....1..
EOR	(X), (Y)	59	1	5	(X) = (X) EOR (Y)	N.....Z.
EOR	A, #i	48	2	2	A = A EOR i	N.....Z.
EOR	A, (X)	46	1	3	A = A EOR (X)	N.....Z.
EOR	A, [d]+Y	57	2	6	A = A EOR ([d]+Y)	N.....Z.
EOR	A, [d+X]	47	2	6	A = A EOR ([d+X])	N.....Z.
EOR	A, d	44	2	3	A = A EOR (d)	N.....Z.
EOR	A, d+X	54	2	4	A = A EOR (d+X)	N.....Z.
EOR	A, !a	45	3	4	A = A EOR (a)	N.....Z.
EOR	A, !a+X	55	3	5	A = A EOR (a+X)	N.....Z.
EOR	A, !a+Y	56	3	5	A = A EOR (a+Y)	N.....Z.
EOR	dd, ds	49	3	6	(dd) = (dd) EOR (ds)	N.....Z.
EOR	d, #i	58	3	5	(d) = (d) EOR i	N.....Z.
EOR1	C, m.b	8A	3	5	C = C EOR (m.b)	C
INC	A	BC	1	2	A++	N.....Z.
INC	X	3D	1	2	X++	N.....Z.
INC	Y	FC	1	2	Y++	N.....Z.
INC	d	AB	2	4	(d)++	N.....Z.
INC	d+X	BB	2	5	(d+X)++	N.....Z.
INC	!a	AC	3	5	(a)++	N.....Z.
INCW	d	3A	2	6	Word (d)++	N.....Z.
JMP	[!a+X]	1F	3	6	PC = [a+X]	....

JMP	!a	5F	3	3	PC = a	....
LSR	A	5C	1	2	Right shift A: 0->high, low->C	N.....ZC
LSR	d	4B	2	4	Right shift (d) as above	N.....ZC
LSR	d+X	5B	2	5	Right shift (d+X) as above	N.....ZC
LSR	!a	4C	3	5	Right shift (a) as above	N.....ZC
MOV	(X)+, A	AF	1	4	(X++) = A (no read)	....
MOV	(X), A	C6	1	4	(X) = A (read)	....
MOV	[d]+Y, A	D7	2	7	([d]+Y) = A (read)	....
MOV	[d+X], A	C7	2	7	([d+X]) = A (read)	....
MOV	A, #i	E8	2	2	A = i	N.....Z.
MOV	A, (X)	E6	1	3	A = (X)	N.....Z.
MOV	A, (X)+	BF	1	4	A = (X++)	N.....Z.
MOV	A, [d]+Y	F7	2	6	A = ([d]+Y)	N.....Z.
MOV	A, [d+X]	E7	2	6	A = ([d+X])	N.....Z.
MOV	A, X	7D	1	2	A = X	N.....Z.
MOV	A, Y	DD	1	2	A = Y	N.....Z.
MOV	A, d	E4	2	3	A = (d)	N.....Z.
MOV	A, d+X	F4	2	4	A = (d+X)	N.....Z.
MOV	A, !a	E5	3	4	A = (a)	N.....Z.
MOV	A, !a+X	F5	3	5	A = (a+X)	N.....Z.
MOV	A, !a+Y	F6	3	5	A = (a+Y)	N.....Z.
MOV	SP, X	BD	1	2	SP = X	....
MOV	X, #i	CD	2	2	X = i	N.....Z.
MOV	X, A	5D	1	2	X = A	N.....Z.
MOV	X, SP	9D	1	2	X = SP	N.....Z.
MOV	X, d	F8	2	3	X = (d)	N.....Z.
MOV	X, d+Y	F9	2	4	X = (d+Y)	N.....Z.
MOV	X, !a	E9	3	4	X = (a)	N.....Z.
MOV	Y, #i	8D	2	2	Y = i	N.....Z.
MOV	Y, A	FD	1	2	Y = A	N.....Z.
MOV	Y, d	EB	2	3	Y = (d)	N.....Z.
MOV	Y, d+X	FB	2	4	Y = (d+X)	N.....Z.
MOV	Y, !a	EC	3	4	Y = (a)	N.....Z.
MOV	dd, ds	FA	3	5	(dd) = (ds) (no read)	....
MOV	d+X, A	D4	2	5	(d+X) = A (read)	....
MOV	d+X, Y	DB	2	5	(d+X) = Y (read)	....
MOV	d+Y, X	D9	2	5	(d+Y) = X (read)	....
MOV	d, #i	8F	3	5	(d) = i (read)	....
MOV	d, A	C4	2	4	(d) = A (read)	....
MOV	d, X	D8	2	4	(d) = X (read)	....
MOV	d, Y	CB	2	4	(d) = Y (read)	....
MOV	!a+X, A	D5	3	6	(a+X) = A (read)	....
MOV	!a+Y, A	D6	3	6	(a+Y) = A (read)	....
MOV	!a, A	C5	3	5	(a) = A (read)	....
MOV	!a, X	C9	3	5	(a) = X (read)	....
MOV	!a, Y	CC	3	5	(a) = Y (read)	....
MOV1	C, m.b	AA	3	4	C = (m.b)	C
MOV1	m.b, C	CA	3	6	(m.b) = C	....
MOVW	YA, d	BA	2	5	YA = word (d)	N.....Z.
MOVW	d, YA	DA	2	5	word (d) = YA (read low only)	....
MUL	YA	CF	1	9	YA = Y * A, NZ on Y only	N.....Z.
NOP		00	1	2	do nothing	....
NOT1	m.b	EA	3	5	m.b = ~m.b	....
NOTC		ED	1	3	C = !C	C
OR	(X), (Y)	19	1	5	(X) = (X) | (Y)	N.....Z.
OR	A, #i	08	2	2	A = A | i	N.....Z.
OR	A, (X)	06	1	3	A = A | (X)	N.....Z.
OR	A, [d]+Y	17	2	6	A = A | ([d]+Y)	N.....Z.
OR	A, [d+X]	07	2	6	A = A | ([d+X])	N.....Z.
OR	A, d	04	2	3	A = A | (d)	N.....Z.
OR	A, d+X	14	2	4	A = A | (d+X)	N.....Z.

OR	A, !a	05	3	4	A = A | (a)	N.....Z.
OR	A, !a+X	15	3	5	A = A | (a+X)	N.....Z.
OR	A, !a+Y	16	3	5	A = A | (a+Y)	N.....Z.
OR	dd, ds	09	3	6	(dd) = (dd) | (ds)	N.....Z.
OR	d, #i	18	3	5	(d) = (d) | i	N.....Z.
OR1	C, /m.b	2A	3	5	C = C | ~(m.b)	C
OR1	C, m.b	0A	3	5	C = C | (m.b)	C
PCALL	u	4F	2	6	CALL $FF00+u	....
POP	A	AE	1	4	A = (++SP)	....
POP	PSW	8E	1	4	Flags = (++SP)	NVPBHIZC
POP	X	CE	1	4	X = (++SP)	....
POP	Y	EE	1	4	Y = (++SP)	....
PUSH	A	2D	1	4	(SP--) = A	....
PUSH	PSW	0D	1	4	(SP--) = Flags	....
PUSH	X	4D	1	4	(SP--) = X	....
PUSH	Y	6D	1	4	(SP--) = Y	....
RET		6F	1	5	Pop PC	....
RET1		7F	1	6	Pop Flags, PC	NVPBHIZC
ROL	A	3C	1	2	Left shift A: low=C, C=high	N.....ZC
ROL	d	2B	2	4	Left shift (d) as above	N.....ZC
ROL	d+X	3B	2	5	Left shift (d+X) as above	N.....ZC
ROL	!a	2C	3	5	Left shift (a) as above	N.....ZC
ROR	A	7C	1	2	Right shift A: high=C, C=low	N.....ZC
ROR	d	6B	2	4	Right shift (d) as above	N.....ZC
ROR	d+X	7B	2	5	Right shift (d+X) as above	N.....ZC
ROR	!a	6C	3	5	Right shift (a) as above	N.....ZC
SBC	(X), (Y)	B9	1	5	(X) = (X)-(Y)-!C	NV..H.ZC
SBC	A, #i	A8	2	2	A = A-i-!C	NV..H.ZC
SBC	A, (X)	A6	1	3	A = A-(X)-!C	NV..H.ZC
SBC	A, [d]+Y	B7	2	6	A = A-([d]+Y)-!C	NV..H.ZC
SBC	A, [d+X]	A7	2	6	A = A-([d+X])-!C	NV..H.ZC
SBC	A, d	A4	2	3	A = A-(d)-!C	NV..H.ZC
SBC	A, d+X	B4	2	4	A = A-(d+X)-!C	NV..H.ZC
SBC	A, !a	A5	3	4	A = A-(a)-!C	NV..H.ZC
SBC	A, !a+X	B5	3	5	A = A-(a+X)-!C	NV..H.ZC
SBC	A, !a+Y	B6	3	5	A = A-(a+Y)-!C	NV..H.ZC
SBC	dd, ds	A9	3	6	(dd) = (dd)-(ds)-!C	NV..H.ZC
SBC	d, #i	B8	3	5	(d) = (d)-i-!C	NV..H.ZC
SET1	d.0	02	2	4	d.0 = 1	....
SET1	d.1	22	2	4	d.1 = 1	....
SET1	d.2	42	2	4	d.2 = 1	....
SET1	d.3	62	2	4	d.3 = 1	....
SET1	d.4	82	2	4	d.4 = 1	....
SET1	d.5	A2	2	4	d.5 = 1	....
SET1	d.6	C2	2	4	d.6 = 1	....
SET1	d.7	E2	2	4	d.7 = 1	....
SETC		80	1	2	C = 1	1
SETP		40	1	2	P = 1	..1.....
SLEEP		EF	1	?	Halts the processor	....
STOP		FF	1	?	Halts the processor	....
SUBW	YA, d	9A	2	5	YA = YA - (d), H on high byte	NV..H.ZC
TCALL	0	01	1	8	CALL [$FFDE]	....
TCALL	1	11	1	8	CALL [$FFDC]	....
TCALL	2	21	1	8	CALL [$FFDA]	....
TCALL	3	31	1	8	CALL [$FFD8]	....
TCALL	4	41	1	8	CALL [$FFD6]	....
TCALL	5	51	1	8	CALL [$FFD4]	....

```
TCALL 6           61    1     8   CALL [$FFD2]                    ........
TCALL 7           71    1     8   CALL [$FFD0]                    ........
TCALL 8           81    1     8   CALL [$FFCE]                    ........
TCALL 9           91    1     8   CALL [$FFCC]                    ........
TCALL 10          A1    1     8   CALL [$FFCA]                    ........
TCALL 11          B1    1     8   CALL [$FFC8]                    ........
TCALL 12          C1    1     8   CALL [$FFC6]                    ........
TCALL 13          D1    1     8   CALL [$FFC4]                    ........
TCALL 14          E1    1     8   CALL [$FFC2]                    ........
TCALL 15          F1    1     8   CALL [$FFC0]                    ........

TCLR1 !a          4E    3     6   (a) = (a)&~A, ZN as for A-(a)   N.....Z.
TSET1 !a          0E    3     6   (a) = (a)|A, ZN as for A-(a)    N.....Z.

XCN   A           9F    1     5   A = (A>>4) | (A<<4)             N.....Z.
```